⊛ OOPs → (Object-Oriented Programming):—

Object-Oriented Programming (OOP) is a programming paradigm that organizes code using objects. It focuses on data & methods rather than procedures & functions.

Note:- C++ is one of the first languages to implement OOP as a core part of the language.

⊛ Why OOP :—

(i) Organizes large codebases
(ii) Makes code modular, reusable, & scalable
(iii) Supports real-world modeling
(iv) Helps in implementing security, abstraction & extensibility

⊛ Basic Building Blocks :—

(1) Class :—

A class is like a blueprint or template.

→ Think of a class as a design of a car - it defines what features the car has (engine, wheels) & what it can do (start, stop).

→ "But this design doesn't create a real car yet - it's just a plan."

Scanned with CamScanner

Date: / / Page:

(ii) Object:-

An object is a real-world version of the class.

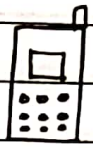
"If a class is a car design, then an object is a real car built from that design.

→ Object is an instance of a class.

(iii) 4 Main Pillars of OOP :-

(1.) Abstraction :-

Hiding the complex implementation details & showing only the necessary features or functionality.



⇒ Use this smartphone without thinking about the how it was made.

(2.) Encapsulation :-

Wrapping data & functions into a single unit (class) & restricting direct access to some of the objects.

↳ The act of putting various components together (in a capsule)



⇒ Laptop is a single entity with screen + camera + keyboard in single box [like a capsule]

Scanned with CamScanner

③ Inheritance :-

Acquiring properties from the parent class.

" Mechanism where one class inherits properties & behaviors (methods) from another class.

Phone ⇒ Smart Phone.

④ Polymorphism :-

One entity can play multiple based on the context or situation.

(Greek word) ← Poly + morphism
↓ ↓
multiple form

Smartphone → Phone (calling)
Smartphone → Calculator.

⑤ Writing a Custom Class :-

```
class Employee{
    int id;      → Attribute 1
    string name; → Attribute 2

}
```

Scanned with CamScanner

Date: / / Page:

① Encapsulation :-

(i) classes & Objects
(ii) Access Specifiers (public, private, protected)
(iii) Constructors & Destructors
(iv) this pointer
(v) Static Members
(vi) Friend functions.

② Abstraction :-

(i) Abstract classes
(ii) Pure virtual functions
(iii) Virtual Function
(iv) Friend functions
(v) Operator Overloading

③ Inheritance :-

(i) Inheritance [Single, Multiple, Multilevel - - -]
(ii) Constructors & Destructors
(iii) Access Specifiers

④ Polymorphism :-

(i) Function Overriding
(ii) Polymorphism (Compile-time & Run-time)

→ (a) Compile-time :- Function overloading, Operator overloading.

→ (b) Run-time :- Virtual Functions.

(iii) Virtual Functions & Pure Virtual Functions
(iv) Operator Overloading

Scanned with CamScanner

Date: / / Page:

⍟ Classes & objects :-

```
#include <iostream>
using namespace std;

→ class Car{
  public:
      string brand;
      int speed;

      void drive(){
          cout<<" Driving"<< brand<<"at"<<speed "<<"km/h";
      }
};
int main(){
→     Car car1; //car1 is an object (instance) of class Car
      car1. brand = " Toyota";
      car1. speed = 100;
      car1. drive();
                  ↳ //output : Driving Toyota at 100 km/h
return 0;

}
```

Scanned with CamScanner

Date: / / Page:

⑧ Access Specifiers :—

Access Specifiers control which part of the program can access the class members (variables & functions).

⑧ Types of Access Specifiers:—

Specifier	Accessible within class	outside class	In Derived class
(i) public	✓	✓	✓
(ii) private	✓	x	x
(iii) protected	✓	x	✓

Note:— By default, class members are private in C++.

Ex:— Class A{
int x; // this is private by default
};

(I) Public:— Open to All
Used when you want members to be accessible from anywhere.

→
```
class Student {
public:
    string name;
  void sayHello(){
      cout<<"Hi, I am"<<name<<endl;
  }};
int main(){ Student S;
        s.name = "Suman"; // Accessible directly
        s.sayHello(); // Accessible directly
return 0;}
```

Scanned with CamScanner

② Private :- Only inside the class

→
```
class BankAccount {
private:
    int balance;
public:
    void setBalance(int b) {
        balance = b;
    }
    int getBalance() {
        return balance;
    }
};

int main() {

→   BankAccount b;
    // b.balance = 1000; → Error: private member

    b.setBalance(1000);

    cout << "Balance :" << b.getBalance();

    return 0;
}
```

Scanned with CamScanner

Date: / / Page:

③ Protected :- Like private, but available to child classes.

→ Used in inheritance. Accessible in the class & in derived classes, but not outside.

→
```
class Parent {
    protected:
        int age = 50;
    };

class child : public Parent {
    public:
        void showAge(){
            cout << "Age is " << age << endl;
        }
    };

→ int main(){

    Child c;
    c.showAge();

    // cout << c.age → Error: protected member

    return 0;
    }
```

Final Notes :-

(i) Use private for data protection (encapsulation)
(ii) Use public to expose necessary behaviour.
(iii) Use protected mainly when working with inheritance.

Scanned with CamScanner

Date: / / Page:

✱ Constructors & Destructors :-

① What is Constructor :-

A constructor is a special function that is automatically called when an object is created from a class. Its main purpose is to initialize data members of the class.

✱ Key points :-

(i) Constructor name is same as the class name.
(ii) No return type, not even void.
(iii) Automatically called when object is created.
(iv) Can be overloaded.

Note :- If you don't write a constructor, C++ provide a default constructor that does nothing.

✱ Types of Constructors :-

① Default Constructor
② Parameterized Constructor
③ Copy Constructor

① Default Constructor :-

(i) No parameters
(ii) Automatically provided by compiler if no constructor is defined

Scanned with CamScanner

Date: / / Page:

Code :-

```
Class Student{
public:
    int id;
    string name;

    // Default constructor
    Student(){
        id = 0;
        name = "Suman";
        cout<<"Default constructor called \n";
    }
void display(){

    cout<<"ID:"<<id<<", Name: "<<name<<endl;
}
};
    int main(){
    Student s1; // Default constructor is called
    s1.display();

return 0; }
```

Scanned with CamScanner

Date: / / Page:

② Parameterized Constructor :-

Takes arguments to initialize data.

→
```
class Student{
public:
    int id;
    string name;

Student(int i, string n){
    id = i;
    name = n;
cout<<" Parameterized constructor called\n";

void display(){
cout<<" ID "<<id<<", Name: "<<name<<endl;

}
};
int main(){

→ Student S2 (101, "Suman"); // Parameterized constructor
    S2.display();

return 0;
}
```

Scanned with CamScanner

Date: / / Page:

③ Copy Constructor :-

Used to create a copy of an existing object.

```
→ Class Student {
public:
    int id;
    string name;

Student (int i, string n) {
    id = i;
    name = n;
}
  // copy constructor

→  Student (const student &s) {
    id = s.id;
    name = s.name;
    cout << " Copy constructor called \n";
  }
void display() {
    cout << " ID: " << id << ", Name: " << name << endl;
  }
};
  int main() {
→   Student s1(102, "Suman");
    Student s2 = s1; // Copy constructor is called
    s2.display();

return 0;
  }
```

Scanned with CamScanner

Date: / / Page:

✱ Destructor:-

① Destructor is A special function with the same name as class but prefixed with ~

② Called automatically when object goes out of scope

③ Used for cleanup like memory release, file close etc.

✱ Key Points:-

① No arguments
② No return type
③ Only one destructor per class
④ Cannot be overloaded.

✱ Code:-

```
class Student {
public:
    Student(){
      cout << " constructor called \n";
      }
   ~ Student(){
      cout << " Destructor called \n";
      }
   };

int main(){
   Student S1; // constructor called
} // Destructor is automatically called at the
                end of scope.
```

Scanned with CamScanner

⑧ Constructor Overloading :-

Constructor overloading means having multiple constructors in the same class with different parameters.

```
class Student {
    string name;
    int age;
public:
    Student() {                    // Default Constructor
    name = "Unknown";
    age = 0;
    }
    Student(string n, int a) {     // Parameterized Constructor
    name = n;
    age = a;
    }
    void display() {
    cout << "Name:" << name << ", Age:" << age << endl;
    }
};
int main() {
Student s1;                        // calls default constructor
Student s2("Suman", 23);           // calls parameterized constructor
s1.display();
s2.display();

return 0;
}
```

Scanned with CamScanner

⭐ this pointer :-

this is keyword in C++. It is a pointer that refers to the current object (the object that is calling the member function).

⭐ Why Use this Pointer :-

(i) To refer to current object's members (especially when parameter names are same as variables.)
(ii) To return the current object from a member function.
(iii) To pass current object as argument.

Code :-

```
class Student{
    string name;
public:
→       student (string name){
            this→name=name; // 'this→name' refers
                               to the member variable
        }
void display(){
    cout<<"Name :"<<this→name<<endl;
}
};
int main(){

→   Student s("Suman");
    s.display();

    return 0;
}
```

Scanned with CamScanner

Date: / / Page:

⭐ Static Data Member :-

(i) It belongs to the class, not to any object.
(ii) Shared across all objects (only 1 copy exists).
(iii) Must be declared inside the class & defined outside.

→
```
# include <iostream>
using namespace std;

class Student {
public:
    static int count;

    Student() { count++; }

    static void showCount() {
        cout << " Total: " << count << endl;
    }
};
```

→
```
int Student :: count = 0;

int main() {

    student a, b;
    Student :: showCount(); // Total : 2

    return 0;
}
```

Scanned with CamScanner

Date: / / Page:

Friend Function :-

A friend function is a non-member function that has the right to access private & protected members of class.

→ Even though, it's not inside the class, the class given it special permission.

Why Use Friend Function:-

By default, private & protected members of a class cannot be accessed outside the class.

→ But is some cases (like operator overloading, two-class operations, or external helper functions), we may want a non-member function to access private data.

That's when we use friend functions.

Syntax :-

```
class ClassName {

    friend returnType functionName(parameters);

};
```

Scanned with CamScanner

Date: / / Page:

④ Key Points :-

Feature	Description
(i) Inside class?	✗ Not a class member
(ii) Access to private members?	✓ Yes (if declared friend)
(iii) Uses this pointer?	✗ No
(iv) Inheritence involved?	✗ No

→ Code :-

```
class Box {
private :
        int length;
public :
        Box(): length(0) {}
// Friend function declaration.
→ friend void setLength(Box &b, int len);
};

// Friend function definition (outside the class)
→ void setLength(Box &b, int len) {
    b.length = len;
    cout << "Length set to: " << b.length << endl;
}
int main() {
    Box b1;
→   setLength(b1, 25); // Accessing private member using
                        friend function
    return 0; }
```

④ When to Use :-

(i) When external functions need access to private data.

(ii) For operator overloading.

Scanned with CamScanner

Date: / / Page:

⁂ Encapsulation :-

Wrapping data (variables) & the code (functions) that operate on the data into a single unit.

In Short :-

Data + functions = class

Access Control = Encapsulation

⁂ Key idea :-

Encapsulation is achieved using:-

(i) A class to wrap data & functions.
(ii) Access specifiers :- private, public, protected

By "By making data member private & using public functions to access/modify them, we encapsulate them."

⁂ Syntax & Example :-

```
class Student {
private :
    string name;
    int age;
public:        // public function to set data
    void setDetails(string n, int a){
        name = n;
        age = a;
    };
```

Scanned with CamScanner

// public function to get data
→ void display() {
cout << "Name: " << name << ", Age: " << age << endl;
}
};

int main() {

→ Student S1;
S1.setDetails ("Suman", 22); // Allowed via public method
S1.display();

//S1.name = "Hacked"; → Not allowed (name is private)

return 0;
}

⭐ output :-

Name: Suman, Age: 22

Scanned with CamScanner

Date: / / Page :

Inheritance :-

Acquiring the properties from the parent class.

or :-

Inheritance is a Mechanism where one class inherits properties & behaviours (methods) from parents (another) class.

Syntax :-

```
class Base {
    // base class members
};
class Derived : access_modifier Base {
    // derived class members
};
```

Types of Inheritance in C++ :-

(i) Single Inheritance
(ii) Multiple Inheritance
(iii) Multilevel Inheritance
(iv) Hierarchical Inheritance
(v) Hybrid Inheritance

Scanned with CamScanner

Date: / / Page:

→ Example:-

```
#include<iostream>
using namespace std;

// Base class
→ class Animal{
  public:
      void eat(){
      cout<<" Eating ..."<<endl;
      }
  };
// Derived class
→ class Dog : public Animal{
  public:
      void bark(){
      cout<<" Barking ..."<<endl;
  }
};

int main(){

→   Dog d;
    d.eat(); // Inherited from Animal
    d.bark(); // Defined in Dog
  return 0;

  }
```

Scanned with CamScanner

Date: / / Page:

① Single Inheritance:-

One class derives from a single base class.
Used where there is a direct "is-a" relationship.

→ Employee
↓
Developer

→
```
class Employee {
public:
    string name; int id;

void displayInfo(){
cout<<"Name:"<<name<<", ID:"<<id<<endl;
};
```

→
```
class Manager : public Employee{
public:
    void manageTeam(){
    cout<<" Managing team..."<<endl;
};
```

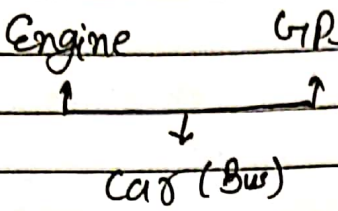
→
```
int main(){
    Manager m;
    m.name = "Suman";
    m.id = 101;
    m.displayInfo();
    m.manageteam();
}
```

Scanned with CamScanner

Date: / / Page:

② Multiple Inheritance:-

A class inherits from two or more base classes.

→ 

→
```
class Engine{
public:
    void startEngine(){
    cout<<" Engine started."<<endl;
}};
```

→
```
class GPS{
public:
    void showLocation(){
    cout<<" Showing GPS location."<<endl;
}};
```

→
```
class Car: public Engine, public GPS{
public:
    void drive(){
        cout<<" Car is driving."<<endl;
}};
```

→
```
int main(){
    Car c;
    c.startEngine();
    c.showLocation();
    c.drive();
}
```

Scanned with CamScanner

Date: / / Page:

③ Multilevel Inheritance:—

A class inherits from another derived class, forming a chain of inheritance.

→ Person → Student → Graduate.

→
```
class Person{
public:
    string name; int roll;

    void showName(){
      cout<<"Name :"<<name<<endl;
    };
```

→
```
class Student : public Person{
 public:
    void showRoll(){
    cout<<" Roll No." <<roll<<endl;
 };
```

→
```
class Graduate: public Student{
public:
    void showDegree(){
    cout<<" Graduation Completed."<<endl;
 };
```

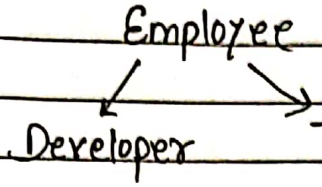
→
```
int main(){
 Graduate g;
    g.name = "Suman";
    g. roll = 22;
     g.showName(); g.showRoll(); g.showDegree();
}
```

Scanned with CamScanner

Date: / / Page:

④ Hierarchical Inheritance:-

→ Multiple derived classes inherit from the same base class.




```
→ class Employee{
public:
   void work(){
   cout<<" employee working...:" <<endl;
};
```

```
→ class Developer : public Employee{
public:
    void code(){
    cout<<" Developer writing code. "<<endl;
};
```

```
→ class Tester : public Employee{
public:
    void test(){
   cout<<" Tester testing application." <<endl;
};
```

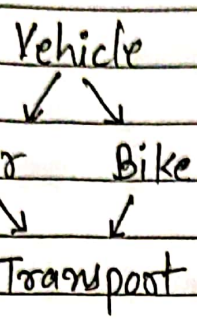
```
→ int main(){
Developer d; Tester t;
d.work(); d.code();
t.work(); t.test();
}
```

Scanned with CamScanner

Page: Date: / /

⑤ Hybrid Inheritance :-

→ Combination of more than one type of inheritance (e.g. Multiple + Hierarchical).



→
```
class Vehicle {
public:
    void start(){
    cout<<" Vehicle starting..."<<endl;
};
```

→
```
class Car: public Vehicle {
public:
    void drive(){
    cout<<" Driving car."<<endl;
};
```

→
```
class Bike : public Vehicle{
public:
    void ride(){
    cout<<" Riding bike."<<endl;
};
```

→
```
class Transport : public Car, public Bike {
public:
    void travel(){
    cout<<" Using transport."<<endl; };
```

Scanned with CamScanner

→ int main(){

Transpost t;

t.start(); // Ambiguity: which Vehicle copy to use?
t.drive();
t.ride();
t.travel();
}

Note:- Without virtual Inheritance, "Transpost" would get two copies of "Vehicle's" members.

→ Problem:- "Vehicle" start function is called twice because two copies exist - one from "car" & one from "Bike".

→ Solution:- With Virtual Inheritance :-

→ class Vehicle {
public:
void start(){
cout<<" Vehicle starting---"<<endl;
};

→ class Car : virtual public Vehicle{
public:
void drive(){
cout<<" Driving car. "<<endl;
}
};

Scanned with CamScanner

→ class Bike : virtual public Vehicle{
public:
    void ride(){
    cout<<"Riding bike."<<endl;
}
};

→ class Transport : public Car, public Bike{
public:
    void travel(){
    cout<<" Using transport."<<endl;
}
};

→ int main(){

    Transport t;
    t.start(); // No ambiguity due to virtual inheritance.
    t.drive();
    t.ride();
    t.travel();

}

→ Use virtual public Vehicle in both Car & Bike to ensure only one shared copy of Vehicle exists.

Scanned with CamScanner

Date: / / Page:

ⓐ Abstraction :-

Abstraction is the process of hiding implementation details & only showing essential features to the user.

→ Example in real life:-

① You drive a car using the steering wheel & pedals.

You don't need to know how the engine, clutch, or fuel injection work.

ⓑ Why Abstraction:-

① To reduce Complexity.
② To improve security (hide sensitive parts)
③ To write flexible, scalable code.
④ To define a general interface for related classes.

ⓒ How to Achieve Abstraction :-

① Encapsulation (Basic level):-

Using access modifiers (private, public) to hide data.

② Abstract classes + Pure Virtual Functions (Advanced level):-

ⓐ Defining a function with virtual void functionName()=0; make it pure virtual.

ⓑ A class with at least one pure virtual function is called an abstract class.

Scanned with CamScanner

(c) You cannot create objects of an abstract class.

(d) Derived classes must implement all pure virtual functions.

(*) Virtual function :-

A virtual function is a function in the base class that you expect to override in derived classes.

→
```
class Base{
public:
    virtual void show(){
    cout<<" Base show"<<endl;
    }
};
```

Note:- It allows runtime polymorphism when using pointers/references.

(*) Pure Virtual Function :-

A pure virtual function is a function that has no body in the base class & must be implemented in derived class.

→
```
class Base{
public:
    virtual void show()=0; // Pure virtual
};
```

Scanned with CamScanner

Date: / / Page:

Example :-

```
→ class Shape {
  public:
      // Pure virtual function
      virtual void draw() = 0;

  void info() { cout << "This is a shape";

  };

→ class Circle : public Shape {
  public:
      void draw() override {
        cout << "Drawing a Circle";
      }
  };
→ class Square : public Shape {
  public:
      void draw() override {
      cout << "Drawing a Square";
      }
  };
→ int main() {
  // Direct object creation of derived classes
    Circle c;
    Square s;

    c.draw(); // output :- Drawing a Circle
    s.draw(); // output :- Drawing a Square.

  }
```

Scanned with CamScanner

Page: Date: / /

⑧ Polymorphism :-

One entity play multiple role based on the situation.

→ The word polymorphism comes from Greek:-

Poly → many
Morph → forms

⑧ Types of Polymorphism :-

① Compile - time Polymorphism
② Run- time Polymorphism

① Compile - time Polymorphism :-

Also called static polymorphism & Early Binding. Behavior is decided at compile time.

→ Example :-

ⓐ Function Overloading :-

Same function name, different parameter list.

ⓑ Operator Overloading :-

Same operator, different meaning for user - defined types.

Scanned with CamScanner

Date: / / Page:

② Runtime Polymorphism :-

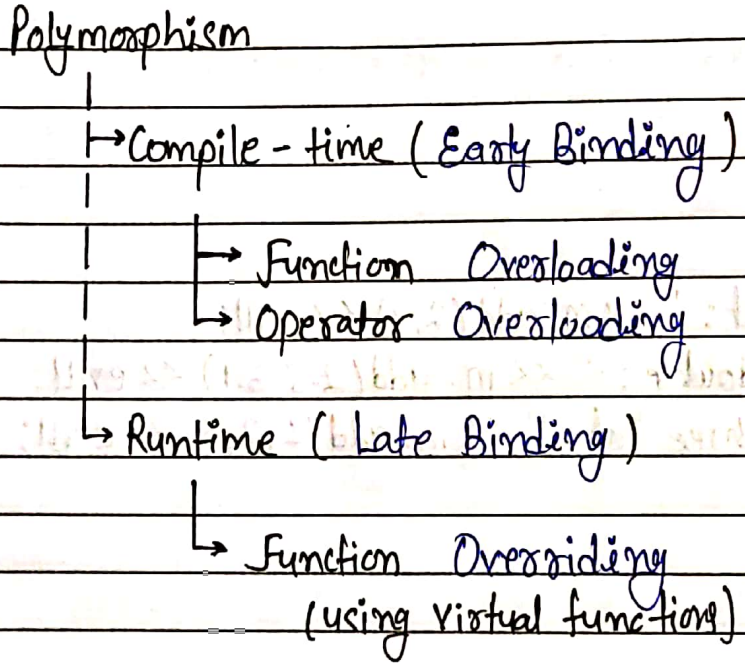
Also called Late Binding / Dynamic Polymorphism. Behavior is decided at runtime using virtual functions & base class pointers/references.

→ Example :-

Function Overriding :-

Derived class provides its own implementation of a base class's virtual function.

③ Diagram Representation :-



Scanned with CamScanner

Page: Date: / /

Example :-

→ Compile-time Polymorphism (Function Overloading):-

```
#include <iostream>
using namespace std;

class Math {
public:
    int add (int a, int b) return a+b;

    double add( double a, double b) return a+b;

    int add( int a, int b, int c) return a+b+c;

};

int main() {
    Math m;

    cout<<"Add int: "<<m.add(2,3)<<endl;
    cout<<" Add double: "<<m.add(2.5,3.1)<<endl;
    cout<<" Add three int :"<<m.add(1,2,3)<<endl;

    return 0;
}
```

Scanned with CamScanner

Date: / / Page:

⭐ Compile-time Polymorphism (Operator Overloading):-

```
#include <iostream>
using namespace std;

class Complex{          // 4+6i → (iota)
public:
    float real, imag;

    Complex(float r, float i){

    real = r; imag = i;}

    // Overload + operator
    Complex operator +(const Complex &c){

    return Complex(real + c.real, imag + c.imag);
    }
};

int main(){
    Complex c1(2.5, 3.5), c2(1.5, 2.5);

    Complex c3 = c1 + c2; // calls overloaded +

    cout << "Result:" << c3.real << " + " << c3.imag << "i" << endl;

    }
```

Real part

Imaginar part

Scanned with CamScanner

Date: / / Page:

Runtime Polymorphism (Function Overriding):-

```
#include <iostream>
using namespace std;

→ class Payment {
public:
    virtual void process() {
    cout << " Processing payment" << endl;
    }
};

→ class CreditCard : public Payment {
 public:
     void process() override {
    cout << " Processing Credit Card Payment" << endl;
}};

→ class Paypal : public Payment {
 public:
     void process() override {
    cout << " Processing PayPal Payment" << endl;
}};

→ int main() {
 Payment * P;
 CreditCard CC;
 PayPal PP;

 P = &CC;  P → process();
 P = &PP;  P → process(); }
```

Scanned with CamScanner

Practice Question :-

(i) Write a program to create a class with private data members & a friend function that can access those members to perform an operation.

(ii) Design a class to present a bank account. Initialize account details using a constructor & display values.

(iii) Create an abstract base class for shapes with a pure virtual function to calculate area. Derive multiple shape classes (like circle, rectangle).

(iv) Implement an employee management system with a base class Employee & derived classes for different roles. (e.g., Manager, Developer). Each derived class should have extra data members & display all details.

(v) Write a program with a base class "Bank" that has a virtual function. Override this function in multiple derived classes to produce different outputs, & demonstrate calling them using base class pointers or references.

Scanned with CamScanner